Nama: Al Jatsiya Profitar Taqwala
NIM: H1D024095
Shift Awal: B
Shift Baru: C

PERTEMUAN 5

1.Source Code

DiagnosaTHT.py > SistemPakarTHT_GUI > setup_ui

```python
import tkinter as tk
from tkinter import messagebox
from tkinter import ttk

class SistemPakarTHT_GUI:
    def __init__(self, root):
        self.root = root
        self.root.title("Sistem Pakar Diagnosa Penyakit THT")
        self.root.geometry("700x600")

        # Data Gejala
        self.gejala = {
            "G1": "Nafas abnormal", "G2": "Suara serak", "G3": "Perubahan kulit",
            "G4": "Telinga penuh", "G5": "Nyeri bicara menelan", "G6": "Nyeri tenggorokan",
            "G7": "Nyeri leher", "G8": "Pendarahan hidung", "G9": "Telinga berdenging",
            "G10": "Airliur menetes", "G11": "Perubahan suara", "G12": "Sakit kepala",
            "G13": "Nyeri pinggir hidung", "G14": "Serangan vertigo", "G15": "Getah bening",
            "G16": "Leher bengkak", "G17": "Hidung tersumbat", "G18": "Infeksi sinus",
            "G19": "Beratbadan turun", "G20": "Nyeri telinga", "G21": "Selaput lendir merah",
            "G22": "Benjolan leher", "G23": "Tubuh tak seimbang", "G24": "Bolamata bergerak",
            "G25": "Nyeri wajah", "G26": "Dahi sakit", "G27": "Batuk", "G28": "Tumbuh dimulut",
            "G29": "Benjolan dileher", "G30": "Nyeri antara mata", "G31": "Radang gendang telinga",
            "G32": "Tenggorokan gatal", "G33": "Hidung meler", "G34": "Tuli",
            "G35": "Mual muntah", "G36": "Letih lesu", "G37": "Demam"
        }

        # Data Penyakit & Aturan
        self.penyakit = {
            "Tonsilitis": ["G37", "G12", "G5", "G27", "G6", "G21"],
            "Sinusitis Maksilaris": ["G37", "G12", "G27", "G17", "G33", "G36", "G29"],
            "Sinusitis Frontalis": ["G37", "G12", "G27", "G17", "G33", "G36", "G21", "G26"],
            "Sinusitis Edmoidalis": ["G37", "G12", "G27", "G17", "G33", "G36", "G21", "G30", "G13", "G26"],
            "Sinusitis Sfenoidalis": ["G37", "G12", "G27", "G17", "G33", "G36", "G29", "G7"],
            "Abses Peritonsiler": ["G37", "G12", "G6", "G15", "G2", "G29", "G10"],
            "Faringitis": ["G37", "G5", "G6", "G7", "G15"],
            "Kanker Laring": ["G5", "G27", "G6", "G15", "G2", "G19", "G1"],
            "Deviasi Septum": ["G37", "G17", "G20", "G8", "G18", "G25"],
            "Laringitis": ["G37", "G5", "G15", "G16", "G32"],
            "Kanker Leher & Kepala": ["G5", "G22", "G8", "G28", "G3", "G11"],
            "Otitis Media Akut": ["G37", "G20", "G35", "G31"],
            "Contact Ulcers": ["G5", "G2"],
            "Abses Parafaringeal": ["G5", "G16"],
            "Barotitis Media": ["G12", "G20"],
            "Kanker Nafasoring": ["G17", "G8"],
            "Kanker Tonsil": ["G6", "G29"],
            "Neuronitis Vestibularis": ["G35", "G24"],
            "Meniere": ["G20", "G35", "G14", "G4"],
            "Tumor Syaraf Pendengaran": ["G12", "G34", "G23"],
            "Kanker Leher Metastatik": ["G29"],
            "Osteosklerosis": ["G34", "G9"],
            "Vertigo Postular": ["G24"]
        }

        self.setup_ui()

    def setup_ui(self):
        # Header Label
        lbl_judul = tk.Label(self.root, text="Sistem Pakar Diagnosa Penyakit THT", font=("Arial", 16, "bold"))
        lbl_judul.pack(pady=10)

        lbl_instruksi = tk.Label(self.root, text="Centang gejala yang Anda alami di bawah ini:", font=("Arial", 10))
        lbl_instruksi.pack(pady=5)
```

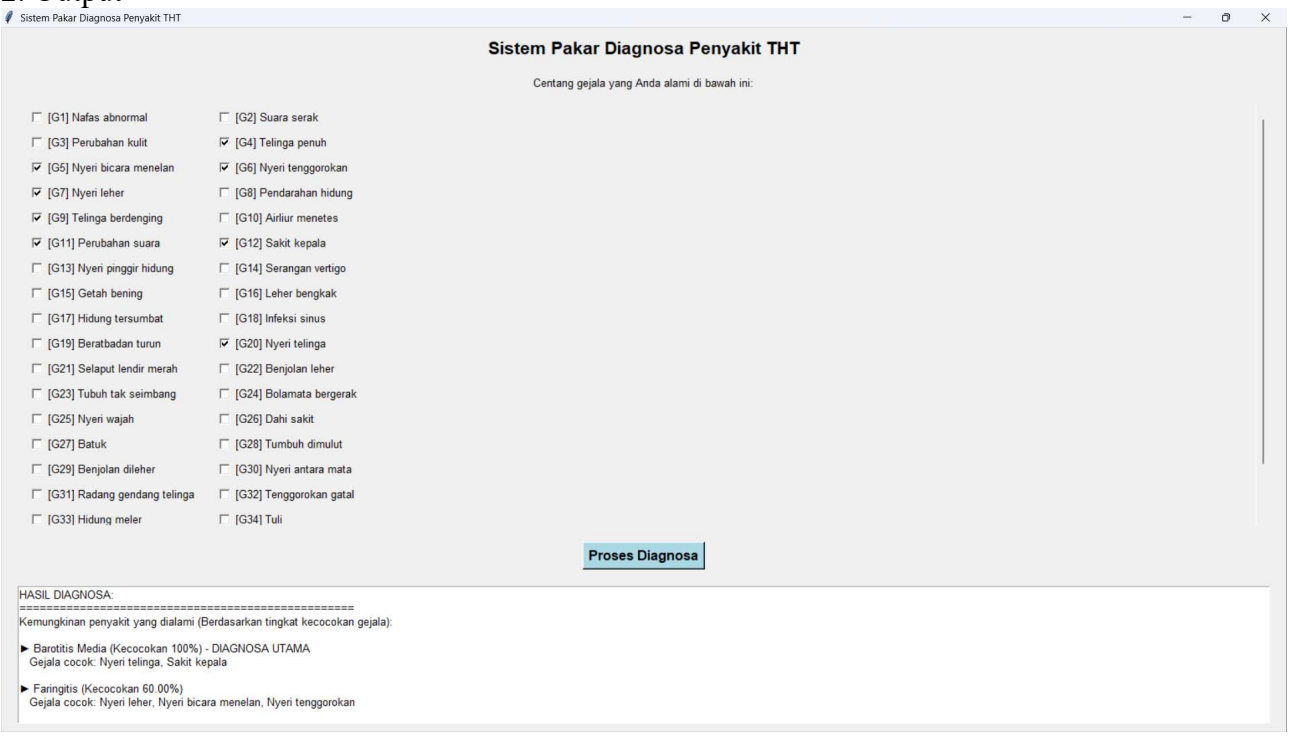
```python
        # Frame untuk Checkbox Gejala dengan Scrollbar
        frame_gejala = tk.Frame(self.root)
        frame_gejala.pack(fill=tk.BOTH, expand=True, padx=20, pady=10)

        canvas = tk.Canvas(frame_gejala)
        scrollbar = ttk.Scrollbar(frame_gejala, orient="vertical", command=canvas.yview)
        scrollable_frame = tk.Frame(canvas)

        scrollable_frame.bind(
            "<Configure>",
            lambda e: canvas.configure(scrollregion=canvas.bbox("all"))
        )

        canvas.create_window((0, 0), window=scrollable_frame, anchor="nw")
        canvas.configure(yscrollcommand=scrollbar.set)

        canvas.pack(side="left", fill="both", expand=True)
        scrollbar.pack(side="right", fill="y")

        # Variabel untuk menyimpan state checkbox
        self.gejala_vars = {}

        # Menampilkan checkbox dalam 2 kolom
        row = 0
        col = 0
        for kode, nama_gejala in self.gejala.items():
            var = tk.BooleanVar()
            self.gejala_vars[kode] = var
            teks_gejala = f"[{kode}] {nama_gejala}"
```

```python
            chk = tk.Checkbutton(scrollable_frame, text=teks_gejala, variable=var, font=("Arial", 10))
            chk.grid(row=row, column=col, sticky="w", padx=10, pady=2)

            col += 1
            if col > 1:
                col = 0
                row += 1

        # Tombol Diagnosa
        btn_diagnosa = tk.Button(self.root, text="Proses Diagnosa", font=("Arial", 12, "bold"), bg="lightblue", command=self.diagnosa)
        btn_diagnosa.pack(pady=10)

        # Text Area untuk Hasil
        self.txt_hasil = tk.Text(self.root, height=10, font=("Arial", 10), state=tk.DISABLED)
        self.txt_hasil.pack(fill=tk.BOTH, expand=False, padx=20, pady=10)

    def diagnosa(self):
        # Ambil gejala yang dicentang
        gejala_dialami = [kode for kode, var in self.gejala_vars.items() if var.get()]

        if not gejala_dialami:
            messagebox.showwarning("Peringatan", "Harap pilih minimal satu gejala terlebih dahulu!")
            return

        # Proses Pencocokan
        hasil_diagnosa = []
        for nama_penyakit, daftar_gejala_penyakit in self.penyakit.items():
            # Hitung jumlah gejala yang cocok
            cocok = set(gejala_dialami).intersection(set(daftar_gejala_penyakit))
            if cocok:
```

```python
                # Persentase = (Gejala yang cocok / Total gejala penyakit) * 100
                persentase = (len(cocok) / len(daftar_gejala_penyakit)) * 100
                hasil_diagnosa.append((nama_penyakit, persentase, cocok))

        # Urutkan berdasarkan persentase tertinggi
        hasil_diagnosa.sort(key=lambda x: x[1], reverse=True)

        # Tampilkan Hasil ke Text Area
        self.txt_hasil.config(state=tk.NORMAL)
        self.txt_hasil.delete(1.0, tk.END)

        self.txt_hasil.insert(tk.END, "HASIL DIAGNOSA:\n")
        self.txt_hasil.insert(tk.END, "="*50 + "\n")

        if hasil_diagnosa:
            self.txt_hasil.insert(tk.END, "Kemungkinan penyakit yang dialami (Berdasarkan tingkat kecocokan gejala):\n\n")
            for nama, persen, cocok in hasil_diagnosa:
                gejala_cocok_str = ", ".join([self.gejala[g] for g in cocok])
                if persen == 100.0:
                    self.txt_hasil.insert(tk.END, f"► {nama} (Kecocokan 100%) - DIAGNOSA UTAMA\n", "utama")
                else:
                    self.txt_hasil.insert(tk.END, f"► {nama} (Kecocokan {persen:.2f}%)\n")
                self.txt_hasil.insert(tk.END, f"   Gejala cocok: {gejala_cocok_str}\n\n")
        else:
            self.txt_hasil.insert(tk.END, "Tidak ada penyakit yang cocok dengan gejala tersebut dalam basis pengetahuan.\n")

        self.txt_hasil.config(state=tk.DISABLED)
```

```python
if __name__ == "__main__":
    root = tk.Tk()
    app = SistemPakarTHT_GUI(root)
    root.mainloop()
```

2. Output



3. Penjelasan

Program ini adalah aplikasi desktop sederhana berbasis Python Tkinter yang berfungsi sebagai sistem pakar untuk mendiagnosis penyakit THT (Telinga, Hidung, Tenggorokan). Cara kerjanya mirip seperti “dokter virtual mini”: pengguna memilih gejala, lalu sistem mencocokkannya dengan pengetahuan yang sudah disimpan.

Di awal, program mengimpor library tkinter, messagebox, dan ttk. Tkinter digunakan untuk membuat tampilan GUI, sementara messagebox dipakai untuk menampilkan peringatan jika pengguna belum memilih gejala.

Masuk ke kelas SistemPakarTHT_GUI, bagian __init__ berperan sebagai fondasi aplikasi. Di sini jendela utama diatur, mulai dari judul hingga ukuran. Lalu ada dua data penting yang menjadi “otak” sistem:

- Data gejala (self.gejala)
  Ini adalah kamus yang berisi kode gejala (G1, G2, dst) dan deskripsinya. Tujuannya supaya sistem bisa mengelola gejala dengan rapi tanpa harus menulis ulang teks panjang setiap saat.
- Data penyakit (self.penyakit)
  Ini adalah basis pengetahuan. Setiap penyakit dikaitkan dengan daftar kode gejala tertentu. Di sinilah konsep sistem pakar bekerja: hubungan antara gejala dan penyakit sudah ditentukan sebelumnya.

Setelah itu, fungsi setup_ui() dipanggil untuk membangun tampilan. Di bagian ini, program membuat label judul, instruksi, dan area utama berisi daftar gejala. Karena jumlah gejala cukup banyak, digunakan kombinasi Canvas + Scrollbar, sehingga pengguna bisa scroll seperti membaca daftar panjang.

Setiap gejala ditampilkan dalam bentuk checkbox. Ketika checkbox dicentang, nilainya disimpan dalam BooleanVar. Semua status checkbox ini dikumpulkan dalam dictionary self.gejala_vars, jadi sistem bisa tahu gejala mana saja yang dipilih.

Tombol “Proses Diagnosa” menjadi pemicu utama. Saat ditekan, fungsi diagnosa() dijalankan. Di sinilah logika inti bekerja.

Pertama, sistem mengambil semua gejala yang dicentang pengguna. Jika tidak ada yang dipilih, muncul peringatan. Kalau ada, sistem mulai mencocokkan dengan setiap penyakit di basis pengetahuan.

Proses pencocokannya cukup elegan: sistem mencari irisan antara gejala yang dipilih pengguna dengan gejala milik suatu penyakit. Semakin banyak yang cocok, semakin tinggi kemungkinan penyakit tersebut. Nilainya dihitung dalam bentuk persentase:

- jumlah gejala cocok dibagi total gejala penyakit
- lalu dikali 100

Hasilnya disimpan, lalu diurutkan dari yang persentasenya paling tinggi. Jadi penyakit yang paling relevan muncul di atas, seperti leaderboard kecil.

Terakhir, hasil ditampilkan di text area. Jika ada kecocokan 100%, sistem memberi label khusus “DIAGNOSA UTAMA”, seolah berkata, “ini kandidat paling kuat.” Selain nama penyakit, sistem juga menampilkan gejala mana saja yang cocok, sehingga pengguna bisa memahami alasannya, bukan sekadar menerima hasil mentah.

Bagian main di bawah hanya berfungsi menjalankan aplikasi: membuat window, menginisialisasi kelas, lalu menjalankan loop utama Tkinter.

Secara keseluruhan, program ini sederhana tapi cukup representatif sebagai sistem pakar berbasis rule. Ia belum menggunakan AI kompleks, tapi logikanya sudah menyerupai proses diagnosis awal: kumpulkan gejala, cocokkan dengan pengetahuan, lalu tampilkan kemungkinan terbaik.